

Lab 18 - Exception Handling

- Due Wednesday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 13 at 6pm - Jul 15 at 11:59pm

Learning Objectives

- Catch exceptions from other functions
- More practice with functions

Completion Requirements

- Submit Lab18.py

Q1: Input Validation

Write a function, `addition()`, that asks the user to enter two numbers separated by space and prints out the sum for the numbers. The users can enter invalid numbers. Your program should prompt the users again in case there is an error. You must inform the user of the exact error. `String.split` function may be helpful.

Do not use `isalpha` function for this question. The following examples show a few example runs.

Example 1: `addition()`

```
Enter two numbers: 2 4
2 + 4 = 6
```



Example 2: `addition()`

```
Enter two numbers: 2 4 3
You entered too many numbers. Try again.
Enter two numbers: qw 45 3
You entered too many numbers. Try again.
Enter two numbers: qw
You entered too few numbers. Try again.
Enter two numbers: qw 45
The first value is not a number. Try again.
Enter two numbers: 50 wq
```

The second value is not a number. Try again.

Enter two numbers: 50 90

50 + 90 = 140

Q2: Catch Exceptions

Write a `listSum()` function that takes a list as input and calculates the sum of the list. The catch is that the list may not have numbers as elements for all the indices. Therefore, the addition operation can fail with a `TypeError`. You need to catch the `TypeError` and return a custom error message to the user as a `String`. The error details in the message should come from the `TypeError` exception. Calling `str()` with the `TypeError` object will be helpful.

Examples:

```
>> print(listSum([1, 2, 3]))
6
>> print(listSum([1, 2, 3, 'a', 4]))
Addition of element 4 raises error : unsupported operand type(s) for +: 'int'
and 'str'
>> print(listSum([1, 2, 3, 4, [1, 2, 3], 4]))
Addition of element 5 raises error : unsupported operand type(s) for +: 'int'
and 'list'
```

Q3: Remove Elements from List

Write a function `remove_evens(lst)` which takes a list as input and removes even numbers from the given list. Do not return a new list. You will be modifying the given list. Calling `List.remove()` function on a list while iterating over the list would break the loop. A possible algorithm is to iterate over the list once and note down the elements to remove in a separate list. Then iterate over the list of items in reverse to remove and remove the items from the main list. All elements in the list do not need to be numbers.

Example 1:

```
>> l1 = [1, 2, 3, 4]
>> remove_evens(l1)
>> print(l1)
[1, 3]
```

Example 2:

```
>> l1 = [1, 2, 3, 4, 2, 4, 10, 89, 43]
>> remove_evens(l1)
>> print(l1)
[1, 3, 89, 43]
```

Example 3:

```
>> l1 = [1, 2, 'Hello', 4, 2, 4, 10, 89, 43]
>> remove_evens(l1)
>> print(l1)
[1, 'Hello', 89, 43]
```

This tool needs to be loaded in a new browser window

Load Lab 18 - Exception Handling in a new window

